LAPORAN TEKNIS

Instalasi dan Konfigurasi PostgreSQL Database Server untuk Sistem CMS Website

PT Terminal Petikemas Surabaya

Nomor Dokumen	TPS-PRIMALINK/DB/2026-001
Versi	1.0
Tanggal	12 Februari 2026
Klasifikasi	Rahasia / Internal
Status	Final

DIPERSIAPKAN UNTUK

PT Terminal Petikemas Surabaya

PT PRIMALINK

Technology Solutions Partner

LEMBAR PENGESAHAN

Judul Dokumen	Laporan Instalasi dan Konfigurasi PostgreSQL Database Server untuk Sistem CMS Website PT Terminal Petikemas Surabaya
Nomor Dokumen	TPS-PRIMALINK/DB/2026-001
Versi	1.0

Disiapkan Oleh	Diperiksa Oleh	Disetujui Oleh
Tim Teknis PT PRIMALINK	**Project Manager** PT PRIMALINK	**PIC IT** PT Terminal Petikemas Surabaya
Tanggal: ___/___/2026	Tanggal: ___/___/2026	Tanggal: ___/___/2026

Riwayat Perubahan Dokumen

Versi	Tanggal	Perubahan	Penulis
0.1	03 Feb 2026	Draft awal — instalasi PostgreSQL	Tim Teknis
0.2	03 Feb 2026	Konfigurasi keamanan dan restore database	Tim Teknis
1.0	12 Feb 2026	Finalisasi dokumen, verifikasi dan validasi	Tim Teknis

DAFTAR ISI

BAB I — PENDAHULUAN 1

1.1 Latar Belakang

1.2 Tujuan

1.3 Ruang Lingkup

1.4 Arsitektur Sistem

BAB II — SPESIFIKASI SERVER 3

2.1 Database Server (172.19.154.92)

2.2 Application Server (172.19.154.93)

2.3 Topologi Jaringan

BAB III — INSTALASI POSTGRESQL 5

3.1 Login ke Server Database

3.2 Persiapan Sistem Operasi

3.3 Instalasi PostgreSQL 16

3.4 Verifikasi Instalasi

BAB IV — KONFIGURASI DATABASE 7

4.1 Konfigurasi Koneksi (postgresql.conf)

4.2 Konfigurasi Autentikasi (pg_hba.conf)

4.3 Konfigurasi User dan Password

4.4 Pembuatan Database

4.5 Restart dan Validasi Service

BAB V — RESTORE DATABASE 10

5.1 Persiapan File Backup

5.2 Proses Restore

5.3 Verifikasi Data

5.4 Struktur Tabel

BAB VI — KONFIGURASI KEAMANAN 13

6.1 Pembatasan Akses Jaringan

6.2 Metode Autentikasi

6.3 Firewall (UFW)

BAB VII — INTEGRASI DENGAN APPLICATION SERVER 15

7.1 Konfigurasi Connection String

7.2 Prisma ORM

7.3 Pengujian Koneksi

BAB VIII — PENGUJIAN DAN VALIDASI 17

8.1 Pengujian Konektivitas

8.2 Pengujian Query

8.3 Pengujian Aplikasi End-to-End

BAB IX — KESIMPULAN DAN REKOMENDASI 19

9.1 Kesimpulan

9.2 Rekomendasi

LAMPIRAN 20

BAB I
PENDAHULUAN

1.1 Latar Belakang

PT Terminal Petikemas Surabaya (TPS) merupakan operator terminal petikemas kelas dunia yang berlokasi di Pelabuhan Tanjung Perak, Surabaya. Sebagai perusahaan yang terus berkembang dan bertransformasi digital, TPS memiliki kebutuhan akan sistem pengelolaan konten website (Content Management System / CMS) yang handal, aman, dan mudah dikelola.

Sistem CMS website TPS yang baru dikembangkan membutuhkan infrastruktur database yang terpisah dari application server untuk memastikan performa optimal, keamanan data, serta kemudahan dalam pengelolaan dan pemeliharaan. Pemisahan ini mengikuti praktik terbaik (best practice) arsitektur multi-tier yang memisahkan presentation layer, application layer, dan data layer.

Dalam rangka implementasi sistem tersebut, PT PRIMALINK selaku mitra teknologi ditugaskan untuk melakukan instalasi, konfigurasi, dan migrasi database PostgreSQL pada server yang telah disediakan oleh TPS di lingkungan jaringan internal perusahaan.

1.2 Tujuan

Dokumen ini disusun dengan tujuan sebagai berikut:

1. Mendokumentasikan seluruh proses instalasi dan konfigurasi PostgreSQL database server pada infrastruktur TPS.
2. Mendokumentasikan proses migrasi (restore) data dari database existing ke server baru.
3. Mendokumentasikan konfigurasi keamanan yang diterapkan pada database server.

4. Memberikan panduan teknis untuk pemeliharaan dan troubleshooting di masa mendatang.
5. Sebagai laporan progres pekerjaan kepada PT Terminal Petikemas Surabaya.

1.3 Ruang Lingkup

Ruang lingkup pekerjaan yang didokumentasikan dalam laporan ini meliputi:

1. Instalasi PostgreSQL 16 pada server database (172.19.154.92).
2. Konfigurasi koneksi, autentikasi, dan keamanan database.
3. Pembuatan user dan database untuk sistem CMS.
4. Restore (migrasi) data dari backup database production.
5. Integrasi dan pengujian koneksi dengan application server (172.19.154.93).
6. Verifikasi dan validasi seluruh data hasil migrasi.

1.4 Arsitektur Sistem

Sistem CMS Website TPS menggunakan arsitektur multi-tier yang memisahkan fungsi aplikasi dan database ke dalam dua server terpisah dalam satu segmen jaringan internal yang sama. Berikut adalah gambaran arsitektur yang diimplementasikan:

```
┌──────────────────────────────────────────────────────────────────────┐
│                      JARINGAN INTERNAL TPS                           │
│                       Subnet: 172.19.154.x                          │
│                                                                      │
│  ┌──────────────────────┐       ┌──────────────────────┐            │
│  │   APPLICATION SERVER  │       │    DATABASE SERVER    │           │
│  │    172.19.154.93      │       │    172.19.154.92      │           │
│  │                       │       │                       │           │
│  │  ┌─────────────────┐  │       │  ┌─────────────────┐  │           │
│  │  │   Docker Engine  │  │       │  │  PostgreSQL 16   │  │           │
│  │  │  ┌────────────┐  │  │       │  │                  │  │           │
│  │  │  │  TPS App   │  │  │ TCP   │  │  Database:       │  │           │
│  │  │  │  (Bun.js)  │──┼──┼───────┼──┼  tps_web         │  │           │
│  │  │  │  Port 3300 │  │  │ 5432  │  │                  │  │           │
│  │  │  └────────────┘  │  │       │  │  16 Tables       │  │           │
│  │  └─────────────────┘  │       │  │  17.773 Records  │  │           │
│  │                       │       │  └─────────────────┘  │           │
│  │  Ubuntu 24.04 LTS     │       │  Ubuntu 24.04 LTS     │           │
│  │  RAM: 8 GB            │       │  RAM: 8 GB            │           │
│  │  Disk: 200 GB         │       │  Disk: 200 GB         │           │
│  │  CPU: 4 Core          │       │  CPU: 4 Core          │           │
│  └──────────────────────┘       └──────────────────────┘            │
│                                                                      │
│  ┌──────────────────────┐                                            │
│  │     USER / ADMIN      │                                            │
│  │   Browser Access      │                                            │
│  │   Port 3300 (HTTP)    │                                            │
│  └──────────────────────┘                                            │
└──────────────────────────────────────────────────────────────────────┘
```

Gambar 1.1 — Diagram Arsitektur Sistem CMS Website TPS

BAB II
SPESIFIKASI SERVER

2.1 Database Server (172.19.154.92)

Server database merupakan server yang didedikasikan untuk menjalankan layanan PostgreSQL. Server ini tidak menjalankan layanan aplikasi lain selain database, sesuai dengan praktik terbaik pemisahan concern.

Parameter	Nilai
Hostname	`tpswebsitedb`
IP Address	172.19.154.92
Sistem Operasi	Ubuntu 24.04.3 LTS (Noble Numbat)
Kernel	Linux 6.8.0-90-generic x86_64
CPU	4 Core
RAM	8 GB
Disk	200 GB (LVM)
Database Engine	PostgreSQL 16.11
Port Database	5432

Tabel 2.1 — Spesifikasi Database Server

2.2 Application Server (172.19.154.93)

Server aplikasi menjalankan sistem CMS dalam container Docker. Aplikasi terhubung ke database server melalui jaringan internal pada port 5432.

Parameter	Nilai
Hostname	`tpswebsiteapp`
IP Address	172.19.154.93
Sistem Operasi	Ubuntu 24.04.3 LTS (Noble Numbat)
Kernel	Linux 6.8.0-90-generic x86_64
CPU	4 Core
RAM	8 GB
Disk	200 GB (LVM)
Docker	Docker 29.2.1
Runtime	Bun 1.1.20 (dalam container)
Port Aplikasi	3300

Tabel 2.2 — Spesifikasi Application Server

2.3 Topologi Jaringan

Kedua server berada dalam satu segmen jaringan internal TPS (subnet 172.19.154.0/24). Komunikasi antar server menggunakan protokol TCP pada port 5432 untuk koneksi database. Jaringan ini merupakan jaringan internal yang tidak memiliki akses langsung ke internet, sehingga meningkatkan keamanan infrastruktur.

Dari	Ke	Port	Protokol	Keterangan
172.19.154.93 (App)	172.19.154.92 (DB)	5432	TCP	Koneksi PostgreSQL
User/Admin	172.19.154.93 (App)	3300	HTTP	Akses Website & CMS

Tabel 2.3 — Koneksi Jaringan Antar Server

BAB III
INSTALASI POSTGRESQL

3.1 Login ke Server Database

Langkah pertama adalah login ke server database (172.19.154.92) melalui SSH. Buka aplikasi terminal (misalnya: PuTTY di Windows, atau Terminal di Mac/Linux).

1 **Login ke Server via SSH**

Ketik perintah berikut di terminal, lalu tekan **Enter**:

```
ssh root@172.19.154.92
```

Jika diminta password, masukkan password server lalu tekan **Enter**. (Password tidak akan terlihat saat diketik — ini normal.)

Jika muncul pertanyaan `Are you sure you want to continue connecting (yes/no)?`, ketik **yes** lalu tekan **Enter**.

Anda berhasil login jika tampilan berubah menjadi: `root@tpswebsitedb:~#`

3.2 Persiapan Sistem Operasi

Sebelum melakukan instalasi PostgreSQL, dilakukan persiapan pada sistem operasi server yang meliputi update paket sistem dan instalasi dependensi yang dibutuhkan.

2 **Update Sistem Operasi**

Ketik perintah berikut lalu tekan **Enter**. Tunggu hingga proses selesai (bisa memakan waktu 1-5 menit):

```
sudo apt update && sudo apt upgrade -y
```

Perintah ini memperbarui daftar paket dan menginstall seluruh update keamanan yang tersedia untuk Ubuntu 24.04 LTS. Proses selesai jika muncul kembali tampilan `root@tpswebsitedb:~#`.

3 **Instalasi Dependensi**

Ketik perintah berikut lalu tekan **Enter**:

```
sudo apt install -y wget curl gnupg2 lsb-release
```

Paket-paket ini diperlukan untuk proses instalasi selanjutnya.

3.3 Instalasi PostgreSQL 16

PostgreSQL versi 16 dipilih karena merupakan versi stabil terbaru yang tersedia di repository resmi Ubuntu 24.04 LTS, dengan dukungan jangka panjang dan fitur-fitur terbaru yang dibutuhkan oleh sistem CMS.

4 **Instalasi PostgreSQL dari Repository Ubuntu**

Ketik perintah berikut lalu tekan **Enter**. Tunggu hingga proses instalasi selesai:

```
sudo apt install -y postgresql postgresql-contrib
```

Paket `postgresql` berisi server database, sedangkan `postgresql-contrib` berisi ekstensi tambahan yang dibutuhkan.

5 **Mengaktifkan Autostart Service**

Ketik dua perintah berikut satu per satu, tekan **Enter** setelah masing-masing perintah:

```
sudo systemctl enable postgresql
sudo systemctl start postgresql
```

> Perintah pertama memastikan PostgreSQL otomatis berjalan ketika server di-restart. Perintah kedua menjalankan PostgreSQL sekarang.

3.4 Verifikasi Instalasi

Setelah instalasi selesai, dilakukan verifikasi untuk memastikan PostgreSQL terinstall dengan benar dan berjalan dengan normal.

> **6** **Cek Versi PostgreSQL**
>
> Ketik perintah berikut lalu tekan **Enter**:
>
> ```
> psql --version
> ```
>
> Harus muncul output seperti ini (versi bisa sedikit berbeda):
>
> ```
> psql (PostgreSQL) 16.11 (Ubuntu 16.11-0ubuntu0.24.04.1)
> ```

> **7** **Cek Status Service PostgreSQL**
>
> Ketik perintah berikut lalu tekan **Enter**:
>
> ```
> sudo systemctl status postgresql
> ```
>
> Harus muncul tulisan `Active: active` berwarna hijau. Tekan huruf **q** untuk keluar dari tampilan status.

> **8** **Cek Cluster PostgreSQL**
>
> Ketik perintah berikut lalu tekan **Enter**:
>
> ```
> pg_lsclusters
> ```
>
> Harus muncul output seperti ini — pastikan Status adalah **online** dan Port adalah **5432**:

```
Ver Cluster Port Status Owner    Data directory
16  main    5432 online postgres /var/lib/postgresql/16/main
```

Item Verifikasi	Expected	Actual	Status
Versi PostgreSQL	16.x	16.11	✅ Pass
Status Service	active	active (exited)	✅ Pass
Cluster Status	online	online	✅ Pass
Port	5432	5432	✅ Pass
Data Directory	/var/lib/postgresql/16/main	/var/lib/postgresql/16/main	✅ Pass

Tabel 3.1 — Hasil Verifikasi Instalasi PostgreSQL

BAB IV

KONFIGURASI DATABASE

4.1 Konfigurasi Koneksi (postgresql.conf)

File konfigurasi utama PostgreSQL perlu diubah agar server database dapat menerima koneksi dari application server. Secara default, PostgreSQL hanya menerima koneksi dari localhost (komputer sendiri). Kita perlu mengubahnya agar application server (172.19.154.93) juga bisa terhubung.

1 **Buka File Konfigurasi dengan Editor Nano**

Ketik perintah berikut lalu tekan **Enter** untuk membuka file konfigurasi:

```
sudo nano /etc/postgresql/16/main/postgresql.conf
```

File akan terbuka di editor **nano**. Anda akan melihat banyak baris teks. Gunakan tombol panah ↑ ↓ untuk scroll ke atas/bawah.

2 **Cari Baris listen_addresses**

Tekan **Ctrl+W** (tombol Ctrl dan W bersamaan) untuk membuka fitur pencarian. Ketik:

```
listen_addresses
```

Lalu tekan **Enter**. Kursor akan berpindah ke baris yang berisi `listen_addresses`. Anda akan melihat baris seperti ini:

```
#listen_addresses = 'localhost'
```

Tanda `#` di depan artinya baris ini dinonaktifkan (komentar).

3 **Ubah Nilai listen_addresses**

Lakukan perubahan berikut pada baris tersebut:

1. Hapus tanda `#` di depan baris (tekan tombol **Delete** atau **Backspace**).
2. Ganti `localhost` menjadi `*` (tanda bintang).

Sebelum:

```
#listen_addresses = 'localhost'
```

Sesudah:

```
listen_addresses = '*'
```

Tanda `*` artinya PostgreSQL menerima koneksi dari semua alamat IP. Keamanan tetap terjaga karena kita akan membatasi IP yang diizinkan di langkah berikutnya.

4 Simpan File dan Keluar dari Editor

Setelah selesai mengedit, lakukan langkah berikut untuk menyimpan:

1. Tekan **Ctrl+O** (tombol Ctrl dan huruf O bersamaan) untuk menyimpan file.
2. Akan muncul pertanyaan nama file — langsung tekan **Enter** untuk konfirmasi.
3. Tekan **Ctrl+X** (tombol Ctrl dan huruf X bersamaan) untuk keluar dari editor nano.

Anda akan kembali ke tampilan terminal normal (`root@tpswebsitedb:~#`).

Tips Editor Nano:

• **Ctrl+W** = Cari teks
• **Ctrl+O** = Simpan file (lalu tekan Enter)
• **Ctrl+X** = Keluar dari editor
• **Panah** ↑ ↓ ← → = Navigasi kursor
• **Ctrl+K** = Hapus satu baris
• Menu shortcut terlihat di bagian bawah layar editor (simbol `^` artinya tombol Ctrl)

Parameter	Nilai	Keterangan
`listen_addresses`	`'*'`	Menerima koneksi dari semua interface
`port`	`5432`	Port standar PostgreSQL
`max_connections`	`100`	Maksimum koneksi bersamaan
`shared_buffers`	`128MB`	Memory untuk shared buffer

Tabel 4.1 — Parameter Konfigurasi PostgreSQL

4.2 Konfigurasi Autentikasi (pg_hba.conf)

File `pg_hba.conf` (Host-Based Authentication) mengontrol siapa yang boleh terhubung ke database, dari mana, dan dengan metode autentikasi apa. Kita perlu menambahkan izin agar application server (172.19.154.93) boleh terhubung.

5 **Buka File pg_hba.conf dengan Editor Nano**

Ketik perintah berikut lalu tekan **Enter**:

```
sudo nano /etc/postgresql/16/main/pg_hba.conf
```

File akan terbuka di editor nano.

6 **Scroll ke Bagian Paling Bawah File**

Tekan **Ctrl+End** (atau tekan tombol panah ↓ berulang kali) sampai Anda berada di baris paling bawah file. Anda akan melihat baris-baris seperti ini di dekat bagian bawah:

```
# IPv4 local connections:
host    all             all             127.0.0.1/32
scram-sha-256
# IPv6 local connections:
host    all             all             ::1/128
scram-sha-256
```

7 **Tambahkan Baris Baru di Paling Bawah**

Pindahkan kursor ke baris paling akhir, lalu tekan **Enter** untuk membuat baris baru. Ketik baris berikut **persis seperti ini** (perhatikan spasi antara setiap kolom):

```
# Akses dari Application Server TPS
host    all    all    172.19.154.93/32    scram-sha-256
```

Baris ini mengizinkan application server (IP 172.19.154.93) untuk terhubung ke database.

8 **Simpan File dan Keluar**

Sama seperti sebelumnya:

1. Tekan **Ctrl+O** untuk menyimpan, lalu tekan **Enter**.
2. Tekan **Ctrl+X** untuk keluar dari editor.

Penjelasan Konfigurasi yang Ditambahkan:

- `host` — Koneksi melalui jaringan TCP/IP
- `all all` — Berlaku untuk semua database dan semua user
- `172.19.154.93/32` — Hanya IP application server yang diizinkan (/32 artinya hanya IP ini saja, bukan range)
- `scram-sha-256` — Metode autentikasi terenkripsi (paling aman yang tersedia di PostgreSQL)

4.3 Konfigurasi User dan Password

Untuk keamanan, password user `postgres` (user utama database) perlu di-set dengan password yang kuat.

9 **Set Password User postgres**

Ketik perintah berikut **dalam satu baris** lalu tekan **Enter**:

```
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD
'TpsDB2026Secure';"
```

Jika berhasil, akan muncul output: `ALTER ROLE`

Catatan Keamanan: Password `TpsDB2026Secure` yang tercantum dalam dokumen ini bersifat rahasia. Distribusi dokumen ini harus dibatasi kepada personel yang berwenang saja. Disarankan untuk mengganti password secara berkala sesuai kebijakan keamanan TPS.

4.4 Pembuatan Database

Database `tps_web` dibuat untuk menyimpan seluruh data sistem CMS website. Ekstensi `uuid-ossp` diaktifkan karena sistem CMS menggunakan UUID sebagai primary key (identitas unik setiap data).

10 **Masuk ke PostgreSQL Console**

Ketik perintah berikut lalu tekan **Enter**:

```
sudo -u postgres psql
```

Tampilan akan berubah menjadi `postgres=#` — ini artinya Anda sudah masuk ke dalam console PostgreSQL.

11 **Buat Database tps_web**

Ketik perintah berikut di console PostgreSQL lalu tekan **Enter**:

```
CREATE DATABASE tps_web;
```

Jika berhasil, akan muncul output: `CREATE DATABASE`

PENTING: Jangan lupa tanda titik koma (`;`) di akhir perintah. Perintah SQL di PostgreSQL harus diakhiri dengan `;`

12 Pindah ke Database tps_web

Ketik perintah berikut lalu tekan **Enter**:

```
\c tps_web
```

Jika berhasil, akan muncul pesan: `You are now connected to database "tps_web" as user "postgres".`

Tampilan berubah menjadi `tps_web=#`

13 Aktifkan Ekstensi UUID

Ketik perintah berikut lalu tekan **Enter**:

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

Jika berhasil, akan muncul output: `CREATE EXTENSION`

14 Verifikasi Ekstensi Terinstall

Ketik perintah berikut lalu tekan **Enter**:

```
\dx
```

Harus muncul daftar yang menampilkan `uuid-ossp` :

```
                           List of installed extensions
    Name      | Version |   Schema   |          Description
-------------+---------+------------
+--------------------------------------
 plpgsql      | 1.0     | pg_catalog | PL/pgSQL procedural
language
 uuid-ossp    | 1.1     | public     | generate universally unique
identifiers
```

15 Keluar dari Console PostgreSQL

Ketik perintah berikut lalu tekan **Enter** untuk keluar dari console PostgreSQL:

```
\q
```

Anda akan kembali ke tampilan terminal normal (`root@tpswebsitedb:~#`).

4.5 Restart dan Validasi Service

Setelah semua konfigurasi selesai, PostgreSQL perlu di-restart agar perubahan diterapkan.

16 **Restart PostgreSQL**

Ketik perintah berikut lalu tekan **Enter**:

```
sudo systemctl restart postgresql
```

Tidak ada output jika berhasil — itu normal.

17 **Pastikan PostgreSQL Berjalan Normal**

Ketik perintah berikut lalu tekan **Enter**:

```
sudo systemctl status postgresql
```

Pastikan muncul tulisan `Active: active`. Tekan huruf **q** untuk keluar dari tampilan status.

Item	Status
listen_addresses = '*'	✅ Configured
Port 5432 aktif	✅ Active
pg_hba.conf — akses 172.19.154.93	✅ Configured
Metode autentikasi scram-sha-256	✅ Configured
Password user postgres	✅ Set
Database tps_web	✅ Created
Ekstensi uuid-ossp	✅ Enabled
Service restart	✅ Restarted

Tabel 4.2 — Checklist Konfigurasi Database

BAB V

RESTORE DATABASE

5.1 Persiapan File Backup

Data dari database production existing perlu dimigrasikan ke server database baru. File backup (dump) dalam format SQL perlu ditransfer ke server database terlebih dahulu.

1 Transfer File Backup ke Server

Perintah ini dijalankan **dari komputer Anda (bukan dari server)**. Buka terminal baru dan ketik:

```
scp tps_web_backup.sql root@172.19.154.92:/tmp/
```

Penjelasan perintah:

- `scp` = Secure Copy — perintah untuk menyalin file melalui SSH
- `tps_web_backup.sql` = nama file backup yang akan ditransfer (file ini harus ada di folder Anda saat ini)
- `root@172.19.154.92:/tmp/` = tujuan: folder `/tmp/` di server database

Jika diminta password, masukkan password server lalu tekan **Enter**. Tunggu hingga transfer selesai (ditandai dengan progress bar 100%).

2 Pastikan File Sudah Ada di Server

Kembali ke terminal yang login ke server database (172.19.154.92). Ketik perintah berikut untuk memastikan file sudah ada:

```
ls -lh /tmp/tps_web_backup.sql
```

> Harus muncul informasi file beserta ukurannya. Jika muncul `No such file or directory`, berarti transfer belum berhasil — ulangi langkah 1.

5.2 Proses Restore

Proses restore dilakukan menggunakan tool `psql` yang membaca file SQL dump dan menjalankan seluruh perintah SQL untuk membuat struktur tabel dan memasukkan data.

> **3** **Restore Database dari File Backup**
>
> Ketik perintah berikut di terminal server database lalu tekan **Enter**:
>
> ```
> sudo -u postgres psql tps_web < /tmp/tps_web_backup.sql
> ```
>
> Penjelasan perintah:
>
> - `sudo -u postgres` = jalankan sebagai user postgres (user database)
> - `psql tps_web` = buka console PostgreSQL di database tps_web
> - `< /tmp/tps_web_backup.sql` = baca dan jalankan isi file backup
>
> **Proses ini akan memakan waktu beberapa menit** tergantung ukuran database. Anda akan melihat banyak output berupa perintah SQL yang dijalankan. Tunggu sampai kembali ke tampilan `root@tpswebsitedb:~#`.
>
> Jika muncul beberapa pesan `ERROR` atau `NOTICE`, itu biasanya normal (misalnya: tabel sudah ada). Yang penting tidak ada pesan `FATAL`.

5.3 Verifikasi Data

Setelah proses restore selesai, dilakukan verifikasi menyeluruh terhadap seluruh tabel dan data yang telah dimigrasikan.

> **4** **Masuk ke Console PostgreSQL**
>
> Ketik perintah berikut lalu tekan **Enter**:

```
sudo -u postgres psql tps_web
```

Tampilan berubah menjadi `tps_web=#`

5 Cek Jumlah Tabel

Ketik perintah berikut lalu tekan **Enter**:

```
\dt
```

Harus muncul daftar 16 tabel. Jika tabel muncul, berarti restore berhasil.

6 Cek Jumlah Data di Tabel Utama

Ketik perintah-perintah berikut **satu per satu**, tekan **Enter** setelah masing-masing:

```
SELECT count(*) FROM content;
```

Hasil yang diharapkan: **17.773** (jumlah konten website)

```
SELECT count(*) FROM file;
```

Hasil yang diharapkan: **5.645** (jumlah file/media)

```
SELECT count(*) FROM structure;
```

Hasil yang diharapkan: **302** (jumlah definisi struktur)

```
SELECT count(*) FROM "user";
```

Hasil yang diharapkan: **9** (jumlah user admin)

PENTING: Pada perintah terakhir, kata `user` harus diapit tanda kutip ganda (`"user"`) karena `user` adalah kata khusus di PostgreSQL.

7 Keluar dari Console PostgreSQL

> Ketik perintah berikut lalu tekan **Enter**:
>
> ```
> \q
> ```
>
> Anda akan kembali ke tampilan terminal normal.

5.4 Struktur Tabel

Database `tps_web` terdiri dari 16 tabel yang saling terelasi. Berikut adalah daftar lengkap tabel beserta ukuran dan jumlah record masing-masing:

No	Nama Tabel	Jumlah Record	Ukuran	Keterangan
1	`content`	17.773	4.160 KB	Konten utama CMS (berita, halaman, dll)
2	`file`	5.645	1.040 KB	Metadata file yang diupload
3	`structure`	302	152 KB	Definisi struktur konten
4	`logs`	3.403	1.696 KB	Log aktivitas pengguna
5	`router`	-	80 KB	Konfigurasi routing URL
6	`structure_folder`	-	64 KB	Folder/kategori struktur
7	`user`	9	32 KB	Data pengguna admin
8	`role`	4	32 KB	Role/peran pengguna
9	`user_session`	-	56 KB	Sesi login pengguna
10	`role_menu`	-	24 KB	Hak akses menu per role
11	`config`	-	32 KB	Konfigurasi sistem CMS
12	`layout`	-	16 KB	Layout halaman website
13	`layout_component`	-	16 KB	Komponen layout
14	`site`	-	16 KB	Konfigurasi site
15	`file_preview`	-	16 KB	Preview/thumbnail file
16	`content_history`	-	16 KB	Riwayat perubahan konten
Total Ukuran Database			**15 MB**	

Tabel 5.1 — Daftar Tabel Database tps_web

Relasi Antar Tabel:

- `content` → `structure` (setiap konten terkait dengan definisi struktur)
- `content` → `file` (konten dapat memiliki file attachment)
- `content` → `content` (self-relation untuk hierarki parent-child)
- `user` → `role` (setiap user memiliki satu role)
- `role_menu` → `structure_folder` (akses menu per role)
- `structure` → `structure_folder` (pengelompokan struktur)

BAB VI

KONFIGURASI KEAMANAN

6.1 Pembatasan Akses Jaringan

Keamanan database diterapkan melalui beberapa lapisan (defense in depth). Lapisan pertama adalah pembatasan akses jaringan melalui konfigurasi `pg_hba.conf` yang hanya mengizinkan koneksi dari IP address yang telah ditentukan.

Sumber	IP Address	Akses	Metode Auth
Localhost	127.0.0.1	✅ Diizinkan	scram-sha-256
Application Server	172.19.154.93	✅ Diizinkan	scram-sha-256
IP Lainnya	*	❌ Ditolak	-

Tabel 6.1 — Matriks Akses Database

6.2 Metode Autentikasi

Metode autentikasi yang digunakan adalah `scram-sha-256`, yang merupakan metode autentikasi paling aman yang tersedia di PostgreSQL 16. Metode ini menggunakan challenge-response mechanism dengan hashing SHA-256, sehingga password tidak pernah dikirimkan dalam bentuk plain text melalui jaringan.

Metode	Keamanan	Digunakan
`trust`	Tidak aman — tanpa password	❌
`password`	Rendah — plain text	❌
`md5`	Sedang — hash MD5	❌
`scram-sha-256`	Tinggi — hash SHA-256	✅

Tabel 6.2 — Perbandingan Metode Autentikasi PostgreSQL

6.3 Firewall (UFW)

Sebagai lapisan keamanan tambahan, disarankan untuk mengkonfigurasi firewall menggunakan UFW (Uncomplicated Firewall) pada server database untuk membatasi port yang dapat diakses.

1 Izinkan Akses Database dari Application Server

Ketik perintah berikut lalu tekan **Enter**:

```
sudo ufw allow from 172.19.154.93 to any port 5432
```

Perintah ini mengizinkan application server (172.19.154.93) mengakses port database (5432).

2 Izinkan Akses SSH (Agar Tidak Terkunci)

Ketik perintah berikut lalu tekan **Enter**:

```
sudo ufw allow ssh
```

PENTING: Pastikan perintah ini dijalankan **sebelum** mengaktifkan firewall, agar Anda tidak terkunci dari server.

3 Aktifkan Firewall

Ketik perintah berikut lalu tekan **Enter**:

```
sudo ufw enable
```

Akan muncul pertanyaan: `Command may disrupt existing SSH connections. Proceed with operation (y|n)?`

Ketik **y** lalu tekan **Enter** untuk mengaktifkan firewall.

4 Verifikasi Status Firewall

Ketik perintah berikut lalu tekan **Enter**:

```
sudo ufw status
```

Harus muncul output seperti ini:

```
Status: active

To                         Action      From
--                         ------      ----
5432                       ALLOW       172.19.154.93
22/tcp                     ALLOW       Anywhere
```

BAB VII
INTEGRASI DENGAN APPLICATION SERVER

7.1 Konfigurasi Connection String

Application server (172.19.154.93) menjalankan aplikasi CMS dalam container Docker. Koneksi ke database dikonfigurasi melalui environment variable `DATABASE_URL` dalam format connection string PostgreSQL.

```
# Format Connection String
postgresql://[user]:[password]@[host]:[port]/[database]

# Konfigurasi aktual
DATABASE_URL="postgresql://
postgres:TpsDB2026Secure@172.19.154.92:5432/tps_web"
```

Connection string ini dikonfigurasi pada dua tempat:

1. **Docker Compose** (`docker-compose.yml`) — sebagai environment variable untuk container.
2. **Prisma ORM** (`app/db/.env`) — untuk ORM database yang digunakan oleh aplikasi.

7.2 Prisma ORM

Aplikasi CMS menggunakan Prisma ORM versi 5.11.0 sebagai database abstraction layer. Prisma menyediakan type-safe query builder dan auto-generated client berdasarkan schema database.

File schema: `app/db/prisma/schema.prisma`

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

7.3 Pengujian Koneksi

Pengujian koneksi dilakukan **dari application server (172.19.154.93)**, bukan dari database server. Buka terminal baru dan login ke application server.

1 **Login ke Application Server**

Buka terminal baru, ketik perintah berikut lalu tekan **Enter**:

```
ssh root@172.19.154.93
```

Masukkan password server jika diminta.

2 **Test Koneksi Jaringan ke Database Server**

Ketik perintah berikut lalu tekan **Enter**:

```
ping -c 3 172.19.154.92
```

Jika berhasil, akan muncul output seperti ini (artinya jaringan terhubung):

```
PING 172.19.154.92: 3 packets transmitted, 3 received, 0% packet
loss
rtt avg: 0.171 ms
```

Jika muncul `100% packet loss`, berarti jaringan bermasalah — hubungi tim jaringan TPS.

3 **Test Port Database Terbuka**

Ketik perintah berikut lalu tekan **Enter**:

```
nc -zv 172.19.154.92 5432
```

Jika berhasil, akan muncul:

```
Connection to 172.19.154.92 5432 port [tcp/postgresql] succeeded!
```

Jika muncul `Connection refused`, berarti PostgreSQL belum berjalan atau konfigurasi `pg_hba.conf` belum benar. Kembali ke BAB IV dan periksa ulang konfigurasi.

4 Test Query Database dari Application Server

Ketik perintah berikut **dalam satu baris** lalu tekan **Enter**:

```
PGPASSWORD=TpsDB2026Secure psql -h 172.19.154.92 -U postgres -d
tps_web -c "SELECT count(*) FROM content;"
```

Penjelasan perintah:

- `PGPASSWORD=TpsDB2026Secure` = password database
- `-h 172.19.154.92` = alamat server database
- `-U postgres` = login sebagai user postgres
- `-d tps_web` = akses database tps_web
- `-c "SELECT ..."` = jalankan query SQL

Jika berhasil, akan muncul jumlah konten (contoh: 17773). Ini membuktikan application server bisa mengakses database.

5 Test Koneksi dari Dalam Docker Container

Jika Docker container sudah berjalan, ketik perintah berikut lalu tekan **Enter**:

```
docker exec tps-app sh -c 'timeout 3 bash -c "echo > /dev/tcp/
172.19.154.92/5432" && echo DB_REACHABLE'
```

Jika berhasil, akan muncul: `DB_REACHABLE`

> Langkah ini bisa dilewati jika Docker container belum diinstall — pengujian di langkah 4 sudah cukup untuk membuktikan koneksi database berfungsi.

BAB VIII

PENGUJIAN DAN VALIDASI

8.1 Pengujian Konektivitas

Test Case	Expected Result	Actual Result	Status
Ping dari App Server ke DB Server	Reply, 0% loss	3/3 received, avg 0.171ms	✅ Pass
TCP port 5432 accessible	Connection succeeded	Connection succeeded	✅ Pass
PostgreSQL auth dari App Server	Authenticated	Authenticated	✅ Pass
Koneksi dari Docker container	DB_REACHABLE	DB_REACHABLE	✅ Pass

Tabel 8.1 — Hasil Pengujian Konektivitas

8.2 Pengujian Query

Test Case	Query	Result	Status
Count content	`SELECT count(*) FROM content`	17.773 rows	✅ Pass
Count files	`SELECT count(*) FROM file`	5.645 rows	✅ Pass
Count structures	`SELECT count(*) FROM structure`	302 rows	✅ Pass
Count users	`SELECT count(*) FROM "user"`	9 rows	✅ Pass
List tables	`\dt`	16 tables	✅ Pass
Database size	`pg_database_size('tps_web')`	15 MB	✅ Pass

Tabel 8.2 — Hasil Pengujian Query Database

8.3 Pengujian Aplikasi End-to-End

Pengujian end-to-end dilakukan dengan mengakses seluruh jenis halaman website untuk memastikan aplikasi dapat membaca data dari database dengan benar.

Halaman	URL	HTTP Status	Size	Status
Homepage	`/`	200	237 KB	✅ Pass
Unduh Dokumen	`/unduh-dokumen`	200	20 KB	✅ Pass
Berita	`/berita/press-release`	200	64 KB	✅ Pass
Admin Dashboard	`/backend/tpsadmin/dashboard`	200	OK	✅ Pass
Admin Edit Content	`/backend/tpsadmin/edit/:id`	200	OK	✅ Pass

Tabel 8.3 — Hasil Pengujian Halaman Website

BAB IX
KESIMPULAN DAN REKOMENDASI

9.1 Kesimpulan

Berdasarkan seluruh proses yang telah dilakukan, dapat disimpulkan bahwa:

1. **Instalasi PostgreSQL 16** pada server database (172.19.154.92) telah berhasil dilakukan dengan status service active dan berjalan normal pada port 5432.
2. **Konfigurasi keamanan** telah diterapkan dengan baik, meliputi pembatasan akses IP hanya dari application server (172.19.154.93) dan penggunaan metode autentikasi `scram-sha-256`.
3. **Migrasi (restore) database** dari backup production telah berhasil dilakukan. Seluruh 16 tabel dengan total 17.773 record konten telah berhasil dimigrasikan dan terverifikasi.
4. **Integrasi** antara application server dan database server telah berfungsi dengan baik, dibuktikan dengan seluruh halaman website yang dapat diakses dan menampilkan data dengan benar.
5. **Pengujian end-to-end** menunjukkan seluruh komponen sistem berfungsi normal, termasuk homepage, halaman berita, halaman download dokumen, dan panel admin CMS.

9.2 Rekomendasi

Untuk menjaga keberlangsungan dan keamanan sistem, direkomendasikan hal-hal berikut:

1. **Backup Berkala** — Mengimplementasikan backup otomatis database menggunakan `pg_dump` dengan jadwal harian, dan menyimpan backup di lokasi terpisah.

```
pg_dump -h 172.19.154.92 -U postgres tps_web > backup_$(date +
%Y%m%d).sql
```

2. **Monitoring** — Memasang monitoring untuk memantau kesehatan database (koneksi aktif, disk usage, query performance).
3. **Rotasi Password** — Mengganti password database secara berkala (minimal setiap 90 hari) sesuai kebijakan keamanan perusahaan.
4. **Update Keamanan** — Menerapkan security patch untuk PostgreSQL dan sistem operasi secara berkala.
5. **Firewall** — Mengaktifkan UFW pada server database sebagai lapisan keamanan tambahan.
6. **SSL/TLS** — Mempertimbangkan aktivasi enkripsi SSL/TLS pada koneksi database untuk keamanan data in-transit.
7. **Connection Pooling** — Jika jumlah koneksi meningkat, mempertimbangkan penggunaan PgBouncer sebagai connection pooler.

LAMPIRAN

Lampiran A — Konfigurasi Lengkap pg_hba.conf

```
# PostgreSQL Client Authentication Configuration File
# TYPE  DATABASE  USER  ADDRESS          METHOD

# Local connections
local   all       postgres                peer
local   all       all                     peer

# IPv4 local connections
host    all       all    127.0.0.1/32     scram-sha-256

# IPv6 local connections
host    all       all    ::1/128          scram-sha-256

# Replication connections
local   replication   all                 peer
host    replication   all  127.0.0.1/32   scram-sha-256
host    replication   all  ::1/128        scram-sha-256

# Application Server access
host    all    all    172.19.154.93/32    scram-sha-256
```

Lampiran B — Prisma Schema (schema.prisma)

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model content {
  id           String   @id
@default(dbgenerated("uuid_generate_v4()")) @db.Uuid
  id_parent    String?  @db.Uuid
  id_structure String?  @db.Uuid
  lang         String?  @default("inherited") @db.VarChar(10)
  text         String?
  status       String?  @default("inherited")
  id_file      String?  @db.Uuid
  created_at   DateTime? @default(now()) @db.Timestamp(6)
  updated_at   DateTime? @db.Timestamp(6)
  // ... (16 models total)
}

// Total: 16 models matching 16 database tables
```

Lampiran C — Docker Compose Configuration

```
services:
  tps-app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: tps-app
    restart: unless-stopped
    ports:
      - "3300:3300"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgresql://
postgres:TpsDB2026Secure@172.19.154.92:5432/tps_web
      - PORT=3300
    volumes:
      - /opt/tps-app/pkgs:/app/pkgs
      - /opt/tps-app/app/srv:/app/app/srv
      - /opt/tps-app/app/db:/app/app/db
      - /opt/tps-app/app/web/deploy:/app/app/web/deploy
      - /opt/tps-data/uploads:/data/files
      - /opt/tps-data/logs:/app/logs
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3300/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
```

Lampiran D — Informasi Kontak

PT PRIMALINK — Tim Teknis	
Perusahaan	PT PRIMALINK
Peran	Technology Solutions Partner
Lingkup	Instalasi, Konfigurasi, dan Pemeliharaan Database Server

Dokumen ini bersifat **RAHASIA** dan hanya untuk penggunaan internal PT Terminal Petikemas Surabaya dan PT PRIMALINK.

TPS-PRIMALINK/DB/2026-001 | Versi 1.0 | 12 Februari 2026